RuleScape

a rule-based optimization for genetic circuit design

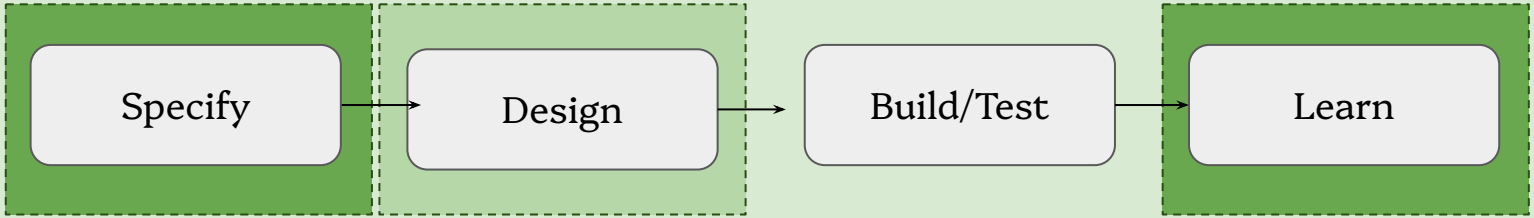
By Juliette, Electra, Meredith, Trieu, and Rafay

Goal and Motivation - Rafay

Let's start with a scenario:

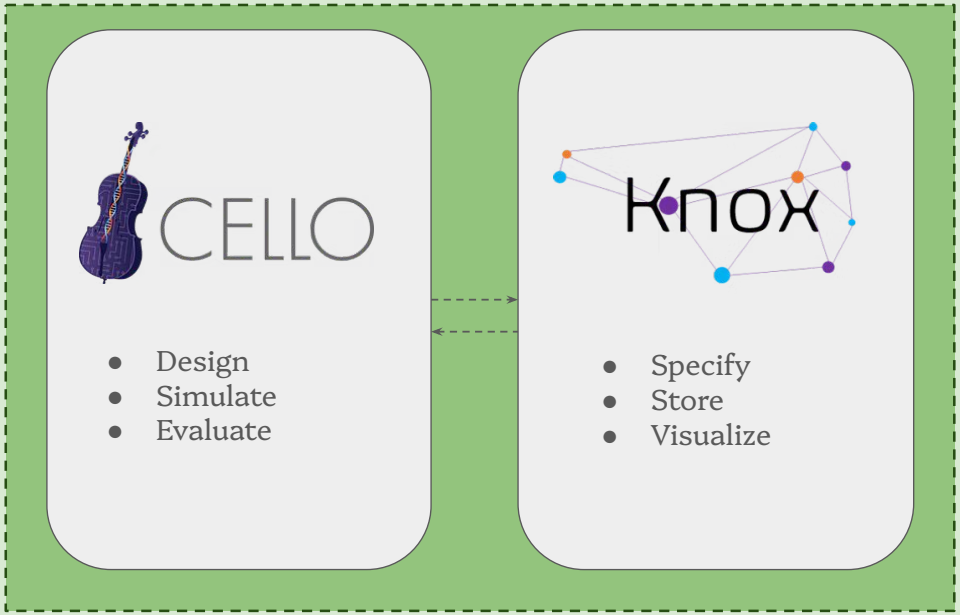
Your PI asks you to make a NOT gate. You know some rules from your personal work, some from other lab members, and you dug some out from lit reviews.

This is the usual flow as you set out on your task



By now, we are all well accounted with this flow...

Goal and Motivation - Rafay



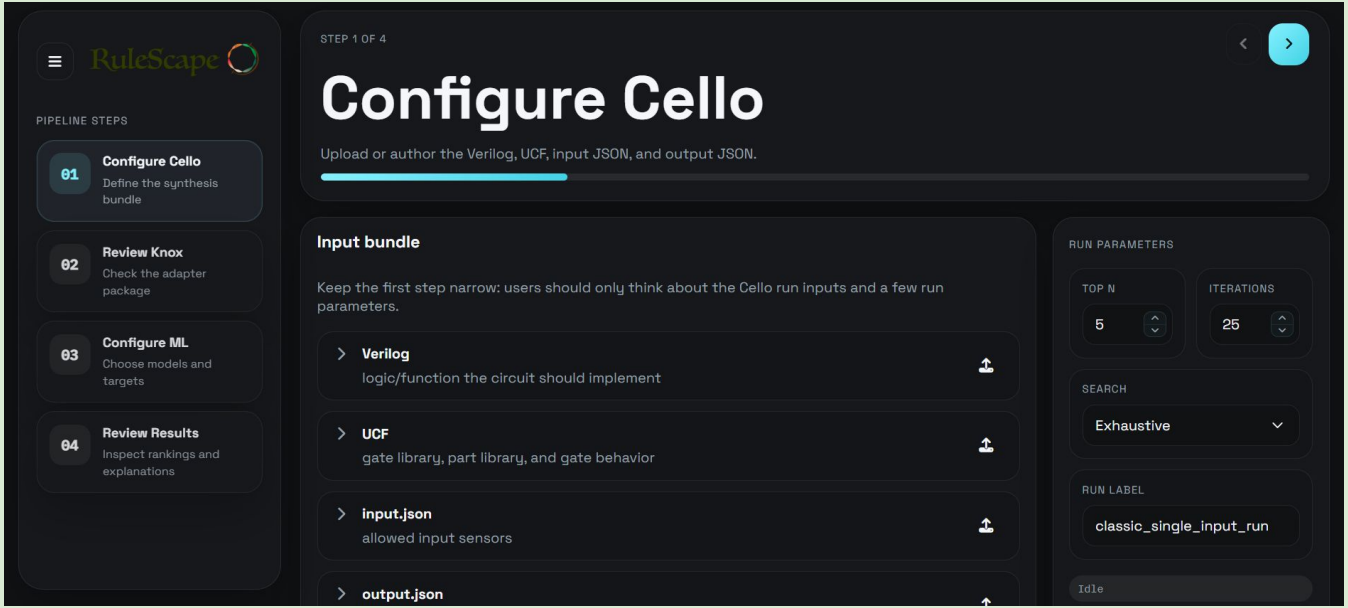
How do we connect these seamlessly?

How do we learn throughout this process?

How do we navigate complex rule-spaces and limit combinatorial explosion?

You can let RuleScape evaluate which rules are the most important for optimizing the expression of the circuit.

Functionality Overview - Meredith



- Develop UCF and CSV files based on data library
- Cello runs and outputs expression scores
- Knox uses these expression scores and their associated designs and evaluates these rules
- ML used to rank these rules based on expression level

Specification Overview - Meredith

Requirements

Inputs

- UCF
- Rules
- Parts
- .mat file
- Verilog file
- Input file
- Output file

All aside from .m must be in .csv for Knox compatibility

Outputs

- Ranked Rules
- Knox and Cello Outputs

Through performance stats. and rule rankings, users can optimize designs and test

Performance

- Node.js + React for GUI
- Python for ML + Adapter Layer

Industry standard framework for optimal performance

Constraints

Scope

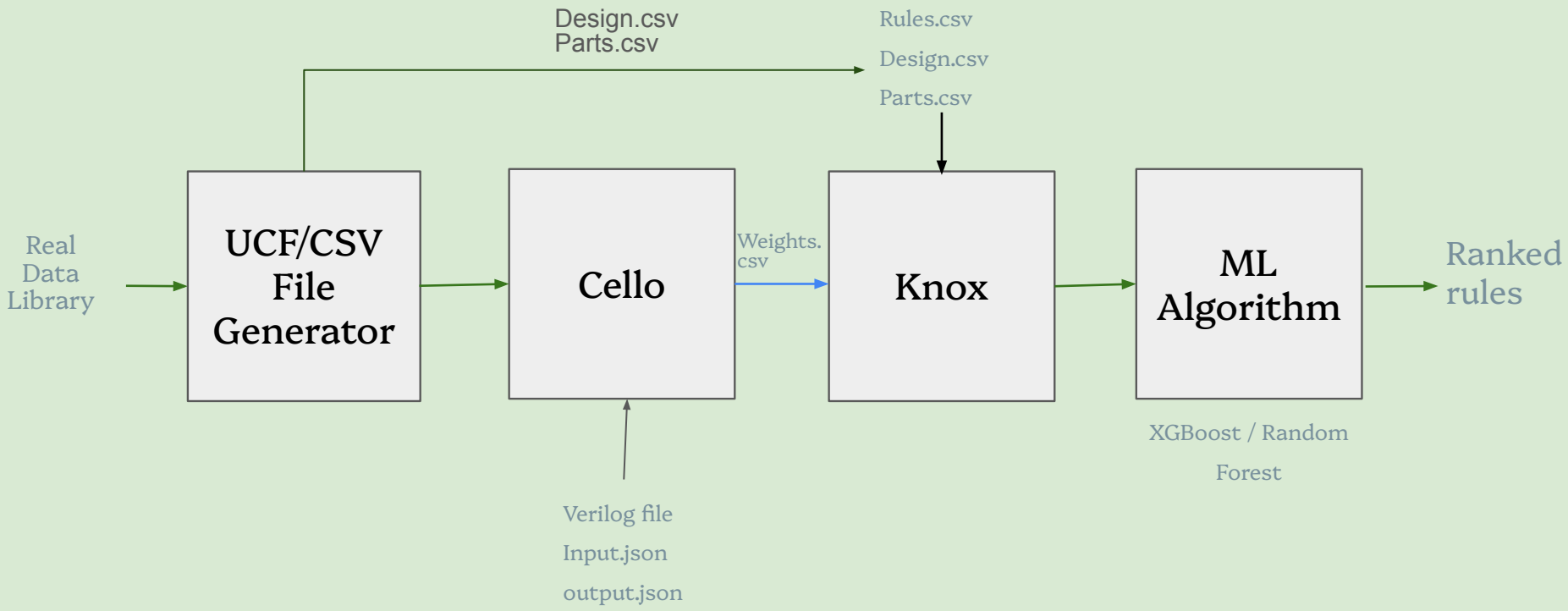
- Yeast Context
- Parts library based on provided data (.mat file)
- Fixed circuit length (8 parts)

System

- Docker + Local servers
- Goldbar syntax for rules

Interoperability between Knox + Cello + RuleScape requires Docker to manage requests between tools

System Diagrams - Meredith



CLASSIC Dataset for Cello and Knox - Juliette

CLASSIC Dataset: real experimental data from single-input library

- Each row → genetic design corresponding to one particular biological construct built from a specific combination of designs
- Each design → 9 design factor column, basal expression, induced expression, fold change and a switch score

Cello Inputs

- User-Constraint File: A structured json that tells which gates exist, what model each gate uses, what parts and structures are associated with the gate, the mathematical response function describing the gate behavior
- Input, output, verilog

Knox Inputs

- Designs
- Parts
- Rules (user-defined)

UCF / input / output / verilog files - Juliette

Step 1: Load CLASSIC dataset

Step 2: Compute switch score

- Rank designs based on switch strength

Step 3: Clean and structure data for Cello

- Choose which rows to export for training dataset (ex: mode=top_100) separated by switch score
- Give each exported design a stable identity (CLASSIC_SI_000001) and convert each row to a Cello gate, model and structure (single-input data acts as a NOT gate)

Step 4: Create parts section - **unique** reusable components

Step 5: Input and output json files with input sensors and output reporters

Stsaaaaaa 6: Verliog → NOT gate for single-input library

Step 7: Inputs to Cello → out of all the gates in this UCF, which one best implements the logical behavior? It outputs a functional score for a selected design

CLASSIC_single_input_top_100.UCF.json


```
[
  {
    "collection": "header",
    "description": "Custom empirical single-input UCF derived from CLASSIC single-input measurements",
    "version": "CLASSIC_single_input_top_100",
    "author": [
      "Juliette Jacques"
    ],
    "organism": "Custom / derived from CLASSIC single-input library"
  },
  {
    "collection": "measurement_std",
    "signal_carrier_units": "AU"
  },
  {
    "collection": "logic_constraints",
    "available_gates": [
      {
        "type": "NOT",
        "max_instances": true
      }
    ]
  },
  {
    "collection": "functions",
    "name": "Hill_response",
    "equation": "ymin + (ymax - ymin) / (1.0 + (x / K)^n)"
  },
  {
```


single_input_ucf_writer.py | make_single_input_ucf.py | App.jsx | install_frontend.sh | Ce

```python
            "equation": "x"
        }
    ]


def build_single_input_ucf(df: pd.DataFrame, version_name="CLASSIC_single_input"):
    """
    Export EXACTLY the dataframe rows provided.
    No internal top-N selection.
    No phenotype spacing filter.
    No hidden downsampling.
    """
    if df.empty:
        raise ValueError("Input dataframe is empty; cannot build UCF.")

    df = df.copy().reset_index(drop=True)

    required_cols = {"basal_expr", "induced_expr", "fold_change", "switch_score"}
    missing = required_cols - set(df.columns)
    if missing:
        raise ValueError(f"Missing required columns for UCF generation: {missing}")

    ucf = []
    ucf.append(make_header(version_name))
    ucf.append(make_measurement_std())
    ucf.append(make_logic_constraints())
    ucf.extend(make_functions())
    ucf.extend(make_parts_from_designs(df))

    for i, row in df.iterrows():
        gate_idx = i + 1
        gate_name = make_gate_name(row, gate_idx)
        ucf.append(make_structure_entry(gate_name))
        ucf.append(make_gate_entry(gate_name))
        ucf.append(make_model_entry(row, gate_name))

    return ucf


def write_ucf_json(ucf, outpath: str):
    with open(outpath, "w", encoding="utf-8") as f:
        json.dump(ucf, f, indent=2)
```

Knox Files - Meredith

Step 1: Load CLASSIC dataset

Step 2: Parse the keys of the .mat file into data frames

- Assign_df: Part assignments for each design
- Exp_df: Expression measurements

Step 3: Build part library

- Maps column prefixes to parts (CDS, Promoter, etc.)
- Generates unique part IDs using design_col_{column number}_{component ID)
- Outputs part_library

Step 4: Build designs library

- Converts each row of assign_df into string of part IDs, separated by commas
- Each row represents one design

Step 5: Export to CSV files

- part_library.csv
- designs.csv

Step 6: Generate a rules .csv file using GOLDBAR

Step 7: Import to Knox

- Import part_library, designs, csv to Knox
- Cello expression outputs are converted into a .csv file and imported into Knox

design								
design_col_0_1	design_col_1_1	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_2	design_col_1_1	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_3	design_col_1_1	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_4	design_col_1_1	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_1	design_col_1_2	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_2	design_col_1_2	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_3	design_col_1_2	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_4	design_col_1_2	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_1	design_col_1_3	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_2	design_col_1_3	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_3	design_col_1_3	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_4	design_col_1_3	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_1	design_col_1_4	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_2	design_col_1_4	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_3	design_col_1_4	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_4	design_col_1_4	design_col_2_1	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_1	design_col_1_1	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_2	design_col_1_1	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_3	design_col_1_1	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_4	design_col_1_1	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_1	design_col_1_2	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_2	design_col_1_2	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_3	design_col_1_2	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_4	design_col_1_2	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_1	design_col_1_3	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_2	design_col_1_3	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_3	design_col_1_3	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1
design_col_0_4	design_col_1_3	design_col_2_2	design_col_3_1	design_col_4_1	design_col_5_1	design_col_6_1	design_col_7_1	design_col_8_1

cds	promoter	operator	M	NI	S	E	T	O	B	R
design_co	design_co	design_co	design_co	design_co	design_co	design_co	design_co	design_co	design_co	design_col_0_1
design_co	design_co	design_co	design_co	design_co	design_co	design_co	design_co	design_co	design_co	design_col_4_1
design_co	design_col_5_3		design_co	design_co	design_col_5_3		design_co	design_co	design_col_6_1:	design_col_4_1
design_co	design_col_5_4		design_co	design_co	design_col_5_4		design_co	design_co	design_col_7_1:	design_col_4_2
design_co	design_col_6_1			design_col_1_1		design_co	design_co	design_co	design_col_5_1:	design_col_4_3
design_co	design_col_6_2			design_col_1_2		design_co	design_co	design_co	design_col_5_2:	design_col_4_4
design_co	design_col_6_3			design_col_4_1			design_col_4_2:desi		design_col_6_1:	design_col_4_5
design_co	design_col_7_1			design_col_4_2			design_col_4_3:desi		design_col_6_2:	design_col_4_6
design_co	design_col_7_2						design_col_4_4:design_col_5_4			
design_co	design_col_7_3						design_col_3_1:design_col_4_1			
design_col_2_3							design_col_3_2:design_col_4_2			
design_col_2_4							design_col_3_3:design_col_4_3			
design_col_2_5							design_col_0_1:design_col_4_1			
design_col_3_1							design_col_0_2:design_col_4_2			
design_col_3_2							design_col_1_1:design_col_4_1			
design_col_3_3							design_col_1_2:design_col_4_2			
design_col_4_1										
design_col_4_2										
design_col_4_3										
design_col_4_4										
design_col_4_5										
design_col_4_6										
design_col_4_7										
design_col_4_8										
design_col_4_9										

id	role
design_col_0_1	cds
design_col_0_2	cds
design_col_0_3	cds
design_col_0_4	cds
design_col_1_1	cds
design_col_1_2	cds
design_col_1_3	cds
design_col_1_4	cds
design_col_2_1	cds
design_col_2_2	cds
design_col_2_3	cds
design_col_2_4	cds
design_col_2_5	cds
design_col_3_1	cds
design_col_3_2	cds
design_col_3_3	cds
design_col_4_1	cds
design_col_4_2	cds
design_col_4_3	cds
design_col_4_4	cds
design_col_4_5	cds
design_col_4_6	cds
design_col_4_7	cds
design_col_4_8	cds
design_col_4_9	cds
design_col_4_10	cds
design_col_4_11	cds
design_col_4_12	cds
design_col_5_1	promoter
design_col_5_2	promoter
design_col_5_3	promoter
design_col_5_4	promoter
design_col_6_1	promoter

ML Pipeline - Electra

Inputs

KnoxStage:
- DesignID
- GroupID
- EvalName

User:
- Models
- Train split
- Top N Rules

Fast API

Receives and Validates ML request

Knox Exctraction

Connects to Knox API

Retrieves designtorule_df

Initiates ML Pipeline

ML Pipeline

Formats dataframe for ML

Splits test/train data

Calls and runs ML model scripts

Returns model summary

GUI Rendering

Receives Response JSON

Displays in UI:
- Model name
- Accuracy
- Top Rules

GUI - Trieu

STEP 1 OF 4

Configure Cello

Upload or author the Verilog, UCF, input JSON, and output JSON.

01 02 03 04

Input bundle

Keep the first step narrow: users should only think about the Cello run inputs and a few run parameters.

Verilog
logic/function the circuit should implement — classic_not_test.v

UCF
gate library, part library, and gate behavior — CLASSIC_single_input_top_10000.UCF.json

input.json
allowed input sensors — CLASSIC_single_input.input.json

output.json
allowed output reporters — CLASSIC_single_input.output.json

```
1 [
2   {
3     "collection": "measurement_std",
4     "signal_carrier_units": "AU"
5   },
6   {
7     "collection": "output_devices",
8     "name": "CLASSIC output reporter",
```

RUN PARAMETERS

CANDIDATES 25 | TOP N 5

SEARCH: Exhaustive

RUN LABEL: classic_single_input_run

CELLO SERVICE: Cello online | RUN STATUS: Ready

Run Cello

STEP 2 OF 4

Review Knox

Choose a Knox bundle, import it into Knox, then evaluate it against Goldbar rules.

01 02 03 04

Knox bundle

Generated | Uploaded

Use the bundle generated from Cello, or upload your own.

designs.csv
ordered selected parts per imported design — pending

part_library.csv
part IDs and roles used during Knox CSV import — pending

weight.csv
optional design scores used for Knox labeling — optional

KNOX PARAMETERS

OUTPUT PREFIX: rulescape_knox

DESIGN GROUP: rulescape_knox_designs

RULE SPACE: rulescape_knox_rules

RULES GROUP: rulescape_knox_rule_group

Rule inputs

STEP 3 OF 4

Configure ML

Set the feature source, score target, and model family for training.

01 02 03 04

Model choice

Recommended
Explainable Boosting Machine
Best fit for interpretable rule features.

Baseline
Random Forest
Strong baseline for tabular feature matrices.

Binary Classification
Decision Tree

ML CONFIGURATION
RUN STATUS: Error

TRAIN/TEST SPLIT: 70%

TOP N FEATURES: 10 (10)

THRESHOLD: 0.5 (0.5)

STEP 4 OF 4

Review Results

Read the ranked candidates, rule signals, and generated artifacts.

01 02 03 04

Ranked designs

Final review should prioritize outcomes: which designs ranked highest and what the model thinks matters.

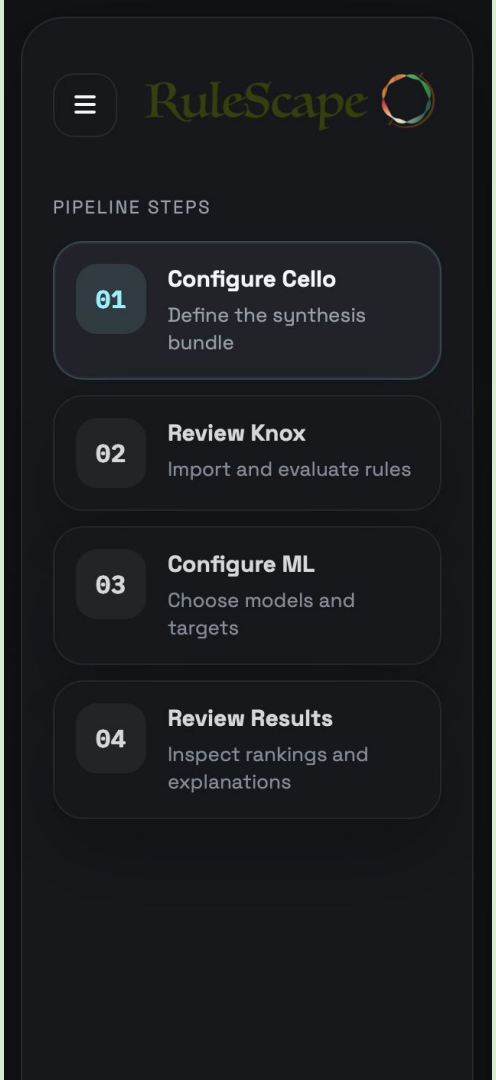
RANK	GATE	SCORE	STATUS
1	Switch_003	1.5052	best
2	Switch_001	1.4767	candidate
3	Switch_005	1.4693	candidate

Top explanations

promoter before operator — 88

Example Interface: Video - Trieu

[Click for User Interface Demo Video]



Limitations - Juliette

Trained model only works well if new datasets are in a similar design space

1	Different parts	✗	✓	A separate UCF and designs, parts and rules inputs would need to be made and input to cello and knox.
2	Different architecture	✗	✓	The ML Model only is trained for the specific datasets used.
3	Different biological system	✗	✓	An improvement for future work would be to automate UCF and rule generation for the future.

Script generating UCF and CSV files are dataframe structure dependent

Electra

Successes

Successfully generated a UCF file and designs from real data

Created an automated pipeline within GUI: Cello->Knox->ML->Rule Ranking

Distribution of work among team

Failures

Organization

Keeping on track with our timeline and finalizing project plans

Shifting directions while maintaining alignment

Deliverables

Scaling with larger and multiple datasets

ML Model Variety

Automated UCF generation for any dataset

Conclusion - Meredith

- RuleScape successfully connects Cello to Knox and ranks rules based on expression scores.
- Future Work:
  - LLM for data parsing - automate input formatting
  - LLM for rule generation - automate GOLDBAR rules
  - More ML models - improve analysis
  - More data - improve training
  - Wet lab validation - real world confirmation